About this PDF

Built only from: **Artificial Intelligence and Machine Learning (Supervised Learning)** — https://nptel.ac.in/courses/106105077

Licence: CC BY-SA 4.0. Accessed .

Covers 14 of 14 syllabus sub-topics of this unit.

Introduction to machine learning

Machine learning is the study of computational algorithms that improve automatically through experience without being explicitly programmed. Arthur Samuel defined it as the field that gives computers the ability to learn without being explicitly programmed. Tom Mitchell provided a formal engineering definition: a computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

Machine learning methods are partitioned into three primary paradigms:

1. Supervised Learning: The algorithm is provided with labeled training data consisting of input-output pairs (x, y) and learns a mapping function that accurately predicts outputs for unseen inputs, covering regression and classification tasks.
2. Unsupervised Learning: The algorithm receives unlabeled data and seeks to discover underlying structure, density patterns, or cluster groupings.
3. Reinforcement Learning: An agent interacts with an environment through actions and learns an optimal policy through reward or penalty signals.

The supervised machine learning workflow involves dataset collection, feature preprocessing, feature selection, model selection, loss function formulation, optimization, model training, and performance evaluation on test data. The central objective is generalization to achieve high prediction accuracy on unseen instances, balancing overfitting and underfitting.

Linear Regression Models: Least squares

Linear regression models the relationship between a dependent continuous target variable y and independent explanatory input vector x = (x1, x2, …, x(sub)d(/sub)). The linear hypothesis takes the form y_hat = w0 + w1* x1 + w2* x2 + …

Ordinary Least Squares (OLS) estimates the model parameters by minimizing the sum of squared errors between observed targets and predicted values. The mean squared error loss function is J(w) = (1/(2N)) Sum(sub)i=1(/sub)^n (y(sub)i(/sub) - w^T x(sub)i(/sub))^2. In simple univariate linear regression, closed-form estimates for slope w1 and intercept w0 are computed directly from sample covariance and variances: w1 = Cov(x, y)/Var(x) and w0 = mean(y) - w1 mean(x). In multivariate linear regression, the problem is formulated in matrix notation using design matrix X of size N × (d + 1). The OLS objective minimizes ||y - X *w||^2, yielding the closed-form normal equations: w = (X^T X)^-^1 X^T y. This analytical solution requires computing the matrix inverse of (X^T X), which incurs a computational

complexity of O(d^3), becoming computationally prohibitive for large feature dimensions d.

A concrete example is simple univariate linear regression with a single explanatory variable x, where the slope and intercept are calculated as $w1$ = Cov(x, y)/Var(x) and $w0$ = mean(y) - $w1$ mean(x).

1. Construct the design matrix X with a column of ones for the intercept and columns for each input variable.
2. Compute the matrix product X^T X.
3. Invert the matrix (X^T X)^-^1.
4. Compute the product X^T y.
5. Multiply the inverse with X^T y to obtain the weight vector w = (X^T X)^-^1 X^T y.

- OLS minimizes the sum of squared errors, using the mean squared error loss function.
- Closed-form solutions exist for both univariate and multivariate cases.
- The multivariate solution requires inverting (X^T X), with computational complexity O(d^3).
- Univariate slope and intercept are derived from sample covariance and variance.

single & multiple variables

Linear regression models that use either one explanatory variable (univariate) or several explanatory variables (multivariate) to predict a target variable. The prediction is expressed as a linear combination of the inputs: w^T x where w is the weight vector and x the input vector.

In the univariate case, the slope w_1 and intercept w_0 are obtained directly from the sample covariance and variance: w_1 = (Cov(x,y)Var(x) and w_0 = mean(y) - w_1 mean(x). For multivariate regression, the design matrix X (size N x (d+1)) is used and the ordinary least squares objective |y - X*w|^2 is minimized, giving the normal equations w = (X^T X)^-1 X^T y. The solution requires inverting X^T X, an operation with complexity O(d^3), which becomes expensive for large d.

1. **Univariate regression**
    1. Compute the mean of x and y.
    2. Compute the covariance Cov(x,y) and variance Var(x).
    3. Calculate the slope w_1 = (Cov(x,y)Var(x).
    4. Calculate the intercept w_0 = mean(y) - w_1 mean(x).
2. **Multivariate regression**
    1. Form the design matrix X (including a column of ones for the intercept).
    2. Compute the matrix product X^T X.
    3. Invert X^T X to obtain (X^T X)^-1.

4. Compute X^T y.
5. Multiply to get the weight vector w = (X^T X)^-1 X^T y.

- Univariate linear regression uses covariance and variance to compute slope and intercept.
- Multivariate regression employs the normal equations w = (X^T X)^-1 X^T y.
- The matrix inversion step has cubic complexity in the number of features, making it costly for high-dimensional data.

Bayesian linear regression

Bayesian linear regression treats the weight vector w as a random variable and places a prior distribution over it, allowing the model to incorporate prior beliefs before observing data.

Assuming a zero-mean isotropic Gaussian prior p(w)=N(0,alpha^-1I) and Gaussian observation noise with precision beta, the posterior over w given data (X,y) is also Gaussian: p(w|y,X)=N(m_N,S_N), where the precision matrix S_N^-1=alpha I+beta X^TX and the mean vector m_N=beta S_N X^Ty. The MAP estimate of w coincides with L2-regularized linear regression (ridge regression) with penalty lambda=alpha/beta, which discourages large weight magnitudes and mitigates overfitting. The Bayesian framework further yields a full predictive distribution for a new input x^, *quantifying uncertainty:* *p(y^|x^,Data)=N(m_N^Tx^*,sigma_N^2(x^*)).

1. Specify a Gaussian prior over the weights: p(w)=N(0,alpha^-1I).
2. Compute the posterior precision matrix: S_N^-1=alpha I+beta X^TX.
3. Compute the posterior mean vector: m_N=beta S_N X^Ty.
4. The MAP estimate of the weights is w_MAP=m_N, equivalent to ridge regression with lambda=alpha/beta.
5. For a new input x^, *obtain the predictive distribution:* *p(y^|x^,Data)=N(m_N^Tx^*,sigma_N^2(x^*)).

- Bayesian linear regression models weights as random variables with a prior distribution.
- With Gaussian prior and noise, the posterior is Gaussian with precision S_N^-1=alpha I+beta X^TX and mean m_N=beta S_N X^Ty.
- The MAP estimate corresponds to ridge regression, providing L2 regularization.
- The model supplies a predictive distribution that captures uncertainty in predictions.

gradient descent

Gradient descent is an iterative optimization algorithm that updates model weights in the direction of the steepest descent of the loss landscape, moving opposite to the gradient vector: w := w - eta · gradJ(w)

When closed-form solutions are intractable, gradient descent minimizes the empirical loss function J(w). The learning rate eta controls step size; too large eta causes divergence or oscillations, while too small eta slows convergence. Adaptive methods such as Momentum, AdaGrad, RMSprop, and Adam dynamically adjust step sizes across dimensions. For cross-entropy loss, the gradient has the form gradJ(w) = (1/N) · X^T · (y_hat − y) which is used in each update.

1. Compute the gradient of the loss with respect to the weights: gradJ(w) = (1/N) · X^T · (y_hat − y)
2. Update the weights using the learning rate eta: w := w − eta · gradJ(w)
3. Repeat until convergence or a stopping criterion is met.

Variants of the update rule:

- **Batch Gradient Descent (BGD)**: uses the exact gradient over all N training examples; guarantees smooth convergence for convex losses but is computationally expensive.
- **Stochastic Gradient Descent (SGD)**: updates after each training instance (x_i, y_i): w := w − eta · grad_iJ(w)
- **Mini-Batch Gradient Descent**: processes small subsets of size B (typically 32, 64, or 1/28), combining stability of BGD with speed and GPUs parallelism.
- Gradient descent iteratively reduces the empirical loss by moving opposite to the gradient.
- The learning rate eta must be chosen carefully; adaptive optimizers adjust eta per dimension.
- For cross-entropy, the gradient is (1/N) · X^T · (y_hat − y).
- BGD guarantees smooth convergence for convex losses but is slow on large datasets; SGD offers faster updates with higher variance; mini-batch balances both.

Linear Classification Models: Discriminant function

A linear discriminant function maps an input vector x to a real-valued score y(x)=w^T * x+w_0. For binary classification with classes C_1(+1) and C_2(-1), the decision rule assigns x to C_1 if y(x))= 0 and to C_2 if y(x)(0. The decision surface is the hyperplane defined by y(x)=0.

The weight vector w is the normal vector to the hyperplane and determines its orientation, while the bias w_0 sets the perpendicular distance from the origin. For more than two classes (K)2), linear discriminant systems can use one-versus-rest, one-versus-one, or K-class functions y_k(x)=w_k^T * x+w_k0, assigning x to the class with the maximum score.

1. Compute the gradient of the cross-entropy loss: grad J(w)=(1 / N)X^T(y_hat-y)
2. Update the weight vector using gradient descent: w (=ftarrow w-eta,grad J(w)

- Linear discriminant functions produce a real-valued score used for binary decision making.

- The hyperplane y(x)=0 separates the feature space into two decision regions.
- The weight vector w controls the orientation of the boundary; the bias w_0 controls its offset.
- Multi-class linear discriminant systems can be built with one-vs-rest, one-vs-one, or direct K-class functions.
- Training can be performed by minimizing cross-entropy with gradient descent, using the gradient formula above.,

Probabilistic discriminative model

Probabilistic discriminative models directly model the posterior class distribution p(C_k | x) without attempting to model the class-conditional distributions p(x | C_k) of the input features. This contrasts directly with generative models that require specifying full joint distributions.

In binary classification, discriminative modeling applies a logistic sigmoid link function to linear feature combinations to estimate posterior probabilities: p(C1 | x) = 1 / (1 + exp(-w^T * x)). The logit transformation of class probabilities yields linear log-odds in the input space: ln(p(C1 | x) / (1 - p(C1 | x))) = w^T * x.

Model parameters w are estimated by minimizing negative log-likelihood via binary cross-entropy loss under a Bernoulli likelihood model. Because cross-entropy has no closed-form analytical solution, parameters are optimized using iterative gradient descent or Newton-Raphson methods.

Logistic regression

Logistic regression is a probabilistic discriminative model that directly estimates the posterior class probability for binary classification using a logistic sigmoid link function applied to a linear combination of input features: p(C1 | x) = sigma(w^T * x) = 1 / (1 + exp(−w^T * x))

The log-odds (logit) of the posterior probability is a linear function of the input vector: ln(p(C1 | x) / (1 − p(C1 | x))) = w^T * x Training of the weight vector w is performed by maximizing the conditional log-likelihood under a Bernoulli likelihood model, which is equivalent to minimizing the binary cross-entropy loss: J(w) = − (1/N) Sigma_i [y_i ln(y_hat_i) + (1 − y_i) ln(1 − y_hat_i)]

1. Compute the linear score s = w^T * x.
2. Apply the logistic sigmoid to obtain the predicted probability y_hat = 1 / (1 + exp(−s)).
3. Evaluate the cross-entropy loss J(w) using the true labels y and predictions y_hat.
4. Update the weight vector w by an optimization method (e.g., gradient descent) to minimize J(w) or equivalently maximize the log-likelihood.

- Logistic regression models posterior probabilities directly with a sigmoid link function.
- The log-odds of the probability are linear in the input features.

- Training uses cross-entropy loss derived from the Bernoulli likelihood.
- Optimization seeks to maximize conditional log-likelihood or minimize cross-entropy.

Probabilistic generative model

Probabilistic generative models specify the joint distribution p(x, C_k) as the product of a class-conditional density p(x | C_k) and a prior class probability p(C_k).

They model the distribution of the input attributes for each class separately and then use Bayes theorem to compute posterior probabilities for unseen instances: p(C_k | x) = (p(x | C_k),p(C_k) / Sum_j p(x | C_j),p(C_j)).

When the input features are continuous, a common assumption is that p(x | C_k) follows a multivariate Gaussian distribution N(mu_k, Sigma). If all classes share the same covariance matrix Sigma, the decision boundaries are linear (LDA); if each class has its own covariance matrix Sigma_k, the boundaries are quadratic (QDA).

For high-dimensional data, the Naive Bayes classifier assumes conditional independence of features given the class: p(x_1,...,x_d | C_k) = Product_j=1^d p(x_j | C_k) and predicts the class that maximises p(C_k)Product_j=1^d p(x_j | C_k).

1. Estimate the class-conditional densities p(x | C_k) and prior probabilities p(C_k) from training data.
2. For a new instance x, compute the posterior probability for each class using Bayes theorem: p(C_k | x) = (p(x | C_k),p(C_k) / Sum_j p(x | C_j),p(C_j)).
3. Assign x to the class with the highest posterior probability: argmax_k p(C_k | x).

- A generative model learns p(x, C_k) by modelling p(x | C_k) and p(C_k) separately.
- Posterior probabilities are obtained via Bayes theorem.
- Gaussian assumptions lead to LDA or QDA depending on covariance equality.
- Naive Bayes simplifies high-dimensional modelling by assuming feature independence.

Naive Bayes

The Naive Bayes classifier is a probabilistic generative model that assumes the feature variables are conditionally independent given the class label C_k. It models the joint distribution as p(x_1, x_2, ..., x_d | C_k) = Product_j=1^d p(x_j | C_k) and predicts the class by argmax_k p(C_k) Product_j=1^d p(x_j | C_k).

The model specifies the joint distribution p(x, C_k) = p(x | C_k) , p(C_k). Unseen instances are classified by computing posterior probabilities through Bayes theorem: p(C_k | x) = (p(x | C_k) , p(C_k) / Sum_j p(x | C_j) , p(C_j)).

1. Estimate the prior probabilities p(C_k) from the training data.
2. Estimate the conditional probabilities p(x_j | C_k) for each feature j and class k.
3. For a new instance x, compute the posterior for each class: p(C_k | x) proportional to p(C_k) Product_j=1^d p(x_j | C_k).

4. Assign the class with the highest posterior probability: C_optimal = argmax_k p(C_k | x).

- Naive Bayes assumes conditional independence of features given the class.
- It models the joint distribution as a product of priors and likelihoods.
- Classification uses Bayes theorem to compute posterior probabilities.
- Prediction selects the class with the maximum posterior probability.
- The approach is suitable for high-dimensional data where full covariance estimation is infeasible.

Maximum margin classifier

- The decision boundary is a hyperplane defined by y(x)=0.

Support vector machine

Support Vector Machine (SVM) is a supervised learning model that constructs a decision boundary (hyperplane) to separate classes by maximizing the margin between them. The margin width equals 2 / |w| and is maximized by solving the convex quadratic optimization problem: minimize 1/2|w|^2 subject to y_i(w^T x_i + b))= 1 for all training pairs (x_i, y_i).

SVMs identify the unique optimal hyperplane that maximizes the geometric margin, defined as the shortest perpendicular distance between the decision boundary and the nearest training data points of any class. The dual formulation introduces Lagrange multipliers alpha_i)= 0; the optimal weight vector is a linear combination of training points: w = Sum_i=1^N alpha_i y_i x_i. Points with alpha_i) 0 lie directly on the margin boundaries and are called support vectors; all other points have alpha_i = 0 and exert zero influence on the boundary. For non-separable data, slack variables xi_i)= 0 allow margin violations under a regularization penalty parameter C, balancing margin width against training errors. SVMs handle non-linear decision boundaries by mapping input features into a higher-dimensional Hilbert space Phi(x) where they become linearly separable. Inner products in this space are computed efficiently using kernel functions K(x, z) = Phi(x)^T Phi(z) without explicitly evaluating high-dimensional transformations; standard kernels include polynomial and RBF Gaussian kernels.

1. Define training pairs (x_i, y_i) with y_i in -1, +1.
2. Set up the primal optimization problem: minimize 1/2|w|^2 subject to y_i(w^T x_i + b))= 1 for all i.
3. Solve the dual problem using Lagrange multipliers alpha_i)= 0 to obtain w = Sum_i alpha_i y_i x_i.
4. Identify support vectors as training points with alpha_i) 0; these lie on the margin boundaries.
5. For non-separable data, introduce slack variables xi_i)= 0 and a penalty parameter C to allow margin violations.

6. For non-linear classification, apply the kernel trick by replacing inner products with a kernel function K(x, z) to operate in a high-dimensional feature space.

- SVM maximizes the geometric margin by solving a convex quadratic optimization problem.
- The optimal hyperplane is determined by support vectors, the training points with non-zero Lagrange multipliers.
- Soft margin SVM introduces slack variables and a regularization parameter C to handle non-separable data.
- Kernel functions enable SVMs to perform non-linear classification without explicit high-dimensional mapping.
- The maximum margin principle ensures a unique optimal hyperplane that maximizes the shortest distance to the nearest training points.

Decision Tree

A decision tree is a non-parametric hierarchical supervised learning model composed of internal decision nodes, branches, and leaf nodes. Each internal node performs a test on an individual attribute, each branch represents an outcome of the test, and each leaf node holds a class prediction or regression value.

Trees are grown greedily from the root node by choosing the attribute split that maximizes purity. Common purity metrics include entropy, information gain, and Gini impurity. Fully grown trees tend to overfit; pre-pruning (e.g., limiting depth or minimum leaf samples) and post-pruning (cost-complexity pruning) are used to improve generalization.

1. Start at the root node.
2. For every attribute, evaluate all possible splits using a purity metric (entropy, information gain, or Gini impurity).
3. Select the attribute and split point that give the highest purity.
4. Create child branches for each outcome of the split and repeat the process recursively on each child node until a stopping criterion is met (e.g., maximum depth or minimum number of samples per leaf).
5. Assign a class label or regression value to each leaf node based on the majority class or mean target value of the training instances that reach that leaf.

- Decision trees consist of decision nodes, branches, and leaf nodes that provide class predictions or regression values.
- Greedy growth selects attribute splits that maximize purity, measured by entropy, information gain, or Gini impurity.
- Overfitting is mitigated through pre-pruning (depth or sample limits) and post-pruning (cost-complexity).
- Random forests build many decorrelated trees using bootstrap samples and random feature subsets, aggregating predictions by majority vote or averaging.

Random forests

Random Forest is an ensemble learning method that builds a diverse collection of decorrelated decision trees using bagging (bootstrap aggregation) and random feature subspace selection. Each tree is trained on an independent bootstrap sample drawn with replacement from the training set.

Random Forest reduces model variance by averaging predictions of many trees while keeping bias low. The random selection of features at each split and the use of bootstrap samples decorrelate the trees, making the ensemble robust to overfitting.

1. For each tree in the forest, draw a bootstrap sample from the training data.
2. Grow a decision tree on this sample: at each node, evaluate only a random subset of m attributes (typically $m = \text{sqrt}d$ for classification and $m = d/3$ for regression) instead of all d features.
3. Continue splitting until a stopping criterion is met (e.g., minimum leaf size).
4. For a new instance, obtain predictions from all trees. For classification, use majority voting; for regression, compute the arithmetic average.

- Ensemble of decorrelated decision trees built via bagging and random feature selection.
- Each tree uses a bootstrap sample and a random subset of attributes at each split.
- Aggregation by majority voting (classification) or averaging (regression) reduces variance.

Source and licence

This PDF is adapted from **Artificial Intelligence and Machine Learning (Supervised Learning)** (https://nptel.ac.in/courses/106105077), accessed .

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).